



Revisão:

Base para desenvolver um programa em C



Prof. Me. Darcy H.F.R. Moreira



ESTRUTURA BÁSICA DO PROGRAMA EM C

Bibliotecas necessárias

Função principal

Chamada do programa

Indica que o programa terminou com sucesso.

C Hello_World.c X

C Hello_World.c > ...

1 → #include <stdio.h>

2

3 → int main() {

4 | printf("Hello, World!\n");

5 | return 0;

6 | }

—

ESTRUTURA BÁSICA DO PROGRAMA EM C

Declarando as variáveis

Definição do **tipo** dos dados a serem armazenados em **variáveis** a serem utilizadas.

TIPOS DE DADOS EM C

int (inteiro)

Inteiros positivos e negativos

```
int quantidade = 10;
```

float (ponto flutuante)

Números decimais (precisão ~ 6 dígitos)

```
float preco = 5.99;
```

double

Números decimais (precisão ~ 15 dígitos)

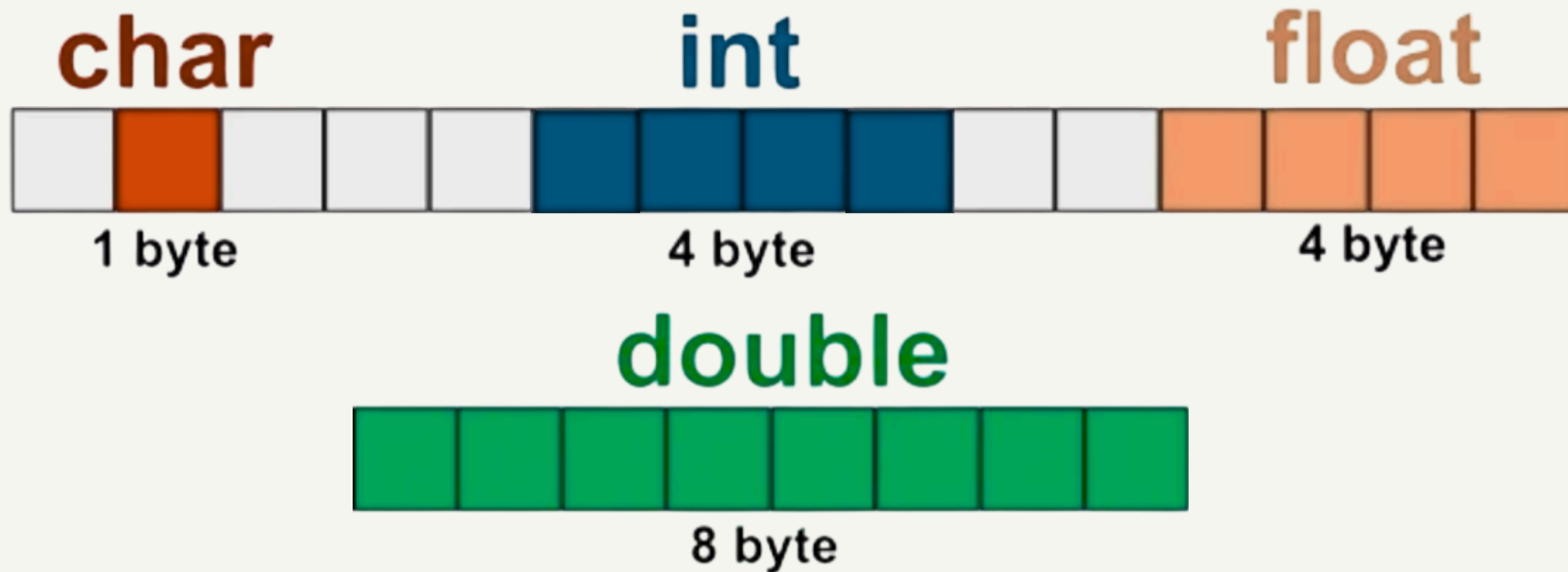
```
double pi = 3.14159;
```

char (Caractere)

Único caractere

```
char letra = 'A';
```

TIPOS DE DADOS EM C



FUNÇÕES DE ENTRADA/SAÍDA

#include <stdio.h>

FUNÇÕES DE ENTRADA/SAÍDA

#include <stdio.h>

Lê os dados inseridos no teclado

$x \rightarrow$ valor de x
 $\&x \rightarrow$ endereço de memória de x

scanf(“%tipo” , $\&x$);

“string” informando o formato do dado

Endereço da variável para onde os dados devem ser lidos

FUNÇÕES DE ENTRADA/SAÍDA

#include <stdio.h>

Lê os dados inseridos no teclado

$x \rightarrow$ valor de x
 $\&x \rightarrow$ endereço de memória de x

scanf(“%tipo”, $\&x$);

“string” informando o formato do dado

Endereço da variável para onde os dados devem ser lidos

Imprime o valor na tela

printf(“valor de x = %tipo”, x);

“string” a ser impressa.

Será substituído pelo valor de x

Valor de x que substituirá o %tipo da string fornecida

FUNÇÕES DE ENTRADA/SAÍDA

Especificador	Tipo	Exemplo
%d	Int	int idade = 20;
%lf	Double(sprintf)	double pi = 3.1415;
%f	Float/double(sprintf)	float altura = 1.75;
%.2f	Float com casas decimais	%.2f = duas casas
%c	char	char letra = 'J';

OPERADORES ARITMÉTICOS

Símbolos usados para operações matemáticas básicas

Operador	Descrição	Exemplo (x = 10, y = 3)	Resultado
+	Soma	$x + y$	13
-	Subtração	$x - y$	7
*	Multiplicação	$x * y$	30
/	Divisão inteira	x / y	3
%	Módulo (resto da divisão)	$x \% y$	1

Divisão de Inteiro por Inteiro resulta em Inteiro:
a parte decimal do resultado é descartada.

OPERADORES ARITMÉTICOS

Para obter um resultado com **casas decimais**, é necessário que pelo **menos um dos valores** da divisão seja do **tipo float**.

```
int a = 15;
int b = 4;

int divisao_inteira = a / b; // Resultado: 3
float divisao_real1 = (float)a / b; // Resultado: 3.75
float divisao_real2 = a / (float)b; // Resultado: 3.75
float divisao_real3 = (float)a / (float)b; // Resultado: 3.75
float divisao_errada = a / b; // Resultado: 3.0
```

OPERADORES RELACIONAIS

Usados para comparar valores, Retornando 1 (true) ou 0 (false)

Operador	Significado	Exemplo (x = 5, y = 10)	Resultado
==	Igual	$x == y$	0
!=	Diferente	$x != y$	1
>	Maior	$x > y$	0
<	Menor	$x < y$	1
>=	Maior ou igual	$x >= y$	0
<=	Menor ou igual	$x <= y$	1

OBS: qualquer valor **diferente** de 0 é **verdadeiro** e **0** é **falso**

OPERADORES LÓGICOS

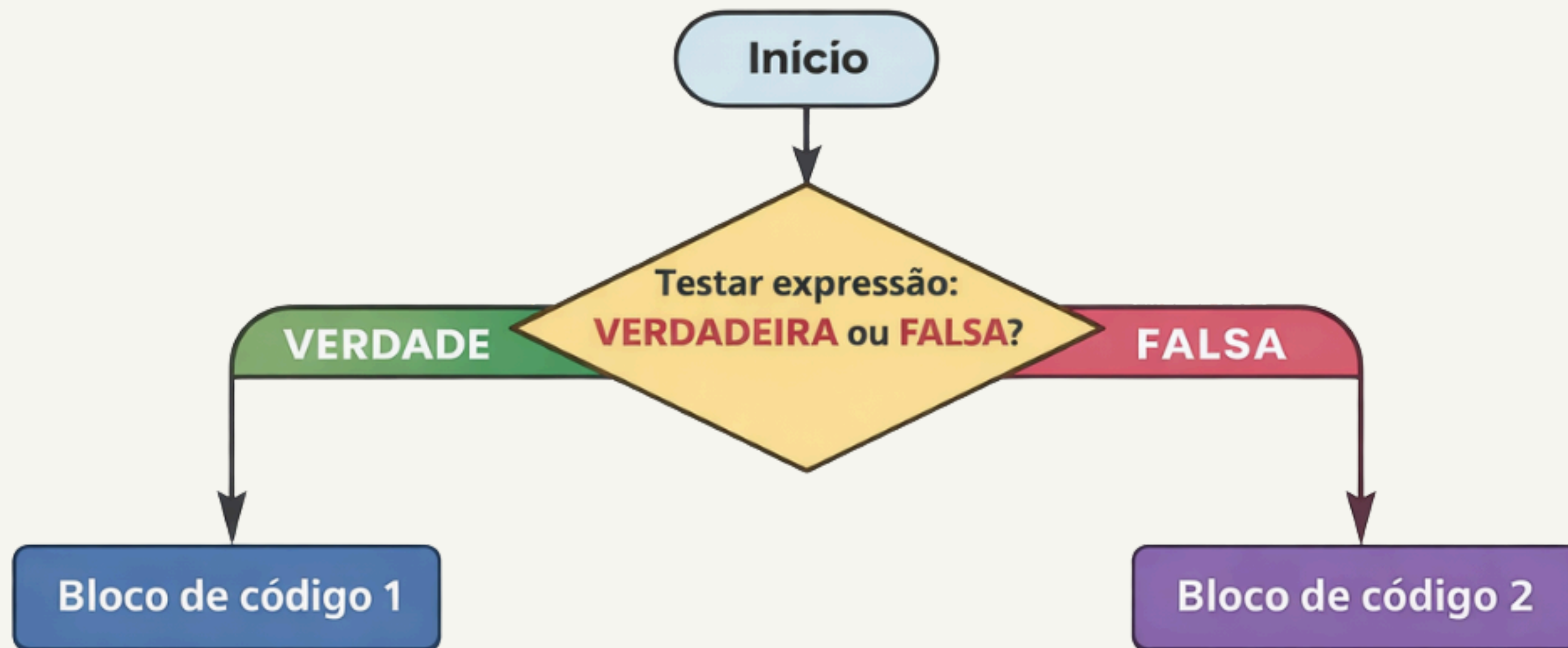
Combinam duas ou mais condições

Operador	Nome	Descrição
&&	E lógico	Verdadeiro se ambas as condições forem verdadeiras
	OU lógico	Verdade de UM for verdade
!	NÃO lógico	Inverte o valor lógico da condição

OBS: qualquer valor **diferente** de 0 é **verdadeiro** e **0** é **falso**

ESTRUTURAS DE CONTROLE DE FLUXO

Permitem ao código tomar **decisões**, executando **blocos** diferentes de código com base em **condições**



ESTRUTURAS DE CONTROLE DE FLUXO

IF, ELSE, ELSE IF

Usado quando há condições lógicas/relacionais.

Switch-Case

Usado quando a escolha depende de valores fixos.

ESTRUTURAS DE CONTROLE DE FLUXO

IF, ELSE, ELSE IF

Usado quando há condições lógicas/relacionais.

Ex: Ler um número e dizer se ele está entre 1 e 10.

```
int num;

printf("Digite um numero inteiro: ");
scanf("%d", &num);

if(num>=1 && num<=10){ //Se o numero estiver entre 1 e 10
    printf("O numero %d esta entre 1 e 10.\n", num);
} else { //Se o numero nao estiver entre 1 e 10
    printf("O numero %d nao esta entre 1 e 10.\n", num);
}
```

ESTRUTURAS DE CONTROLE DE FLUXO

Switch-Case

Há opções fixas possíveis para uma mesma variável. Evita muitos if-else.

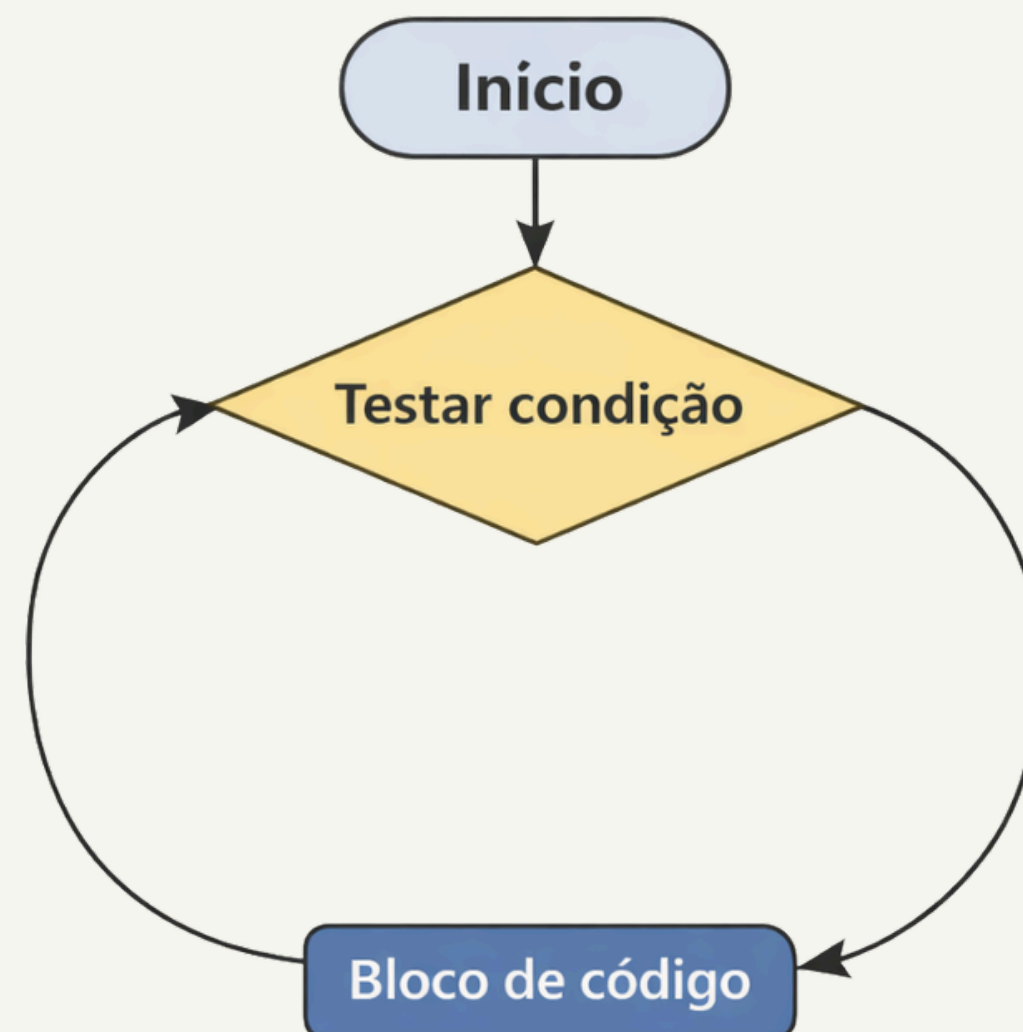
Ex: Mostrar uma mensagem de acordo com o número digitado.

```
int op;
printf("Escolha uma opcao (1 ou 2):\n");
scanf("%d", &op);

switch(op){
    case 1:
        printf("Voce escolheu a opcao 1.\n");
        break;
    case 2:
        printf("Voce escolheu a opcao 2.\n");
        break;
    default:
        printf("Opcao invalida.\n");
}
```

LAÇOS DE REPETIÇÃO

Quando quero executar um bloco várias vezes.



LAÇOS DE REPETIÇÃO

for

Quando **sabemos** quantas vezes o laço vai se repetir.

```
for(inicializacao; condicao; incremento)
{
    //bloco de codigo a ser repetido
}
```

**Ex: Imprimir os
números de 1 a 10:**

```
int i;
for(i=1; i<=10; i++){
    printf("%d\n", i);
}
```

LAÇOS DE REPETIÇÃO

while

- Quando **não** sabemos quantas vezes o laço vai se repetir.
- Quando a repetição depende de uma **condição**.

```
while(condicao)
{
    //bloco de codigo a ser repetido enquanto a condicao for verdadeira
}
```

Ex: Imprimir os números de 1 a 10:

```
int i = 1;
while(i <= 10){
    printf("%d\n", i);
    i++;
}
```

LAÇOS DE REPETIÇÃO

do...while

- é usado quando **não** sabemos quantas vezes o laço vai se repetir.
- Quando a repetição depende de uma **condição**.
- **Garanto que o bloco será executado pelo menos 1 vez**

```
do
{
    //bloco de codigo a ser repetido enquanto a condicao for verdadeira
} while(condicao);
```

Ex: Imprimir os números de 1 a 10:

```
int i = 1;
do{
    printf("%d\n", i);
    i++;
} while(i <= 10);
```

LAÇOS DE REPETIÇÃO

break

- O laço é interrompido automaticamente

continue

- Pula a repetição atual e vai para a próxima iteração

Ex: Imprimir os números de 1 a 5:

```
for(i=1; i<=10; i++){  
    if(i == 6){  
        break; //Interrompe  
    }  
    printf("%d\n", i);  
}
```

1
2
3
4
5

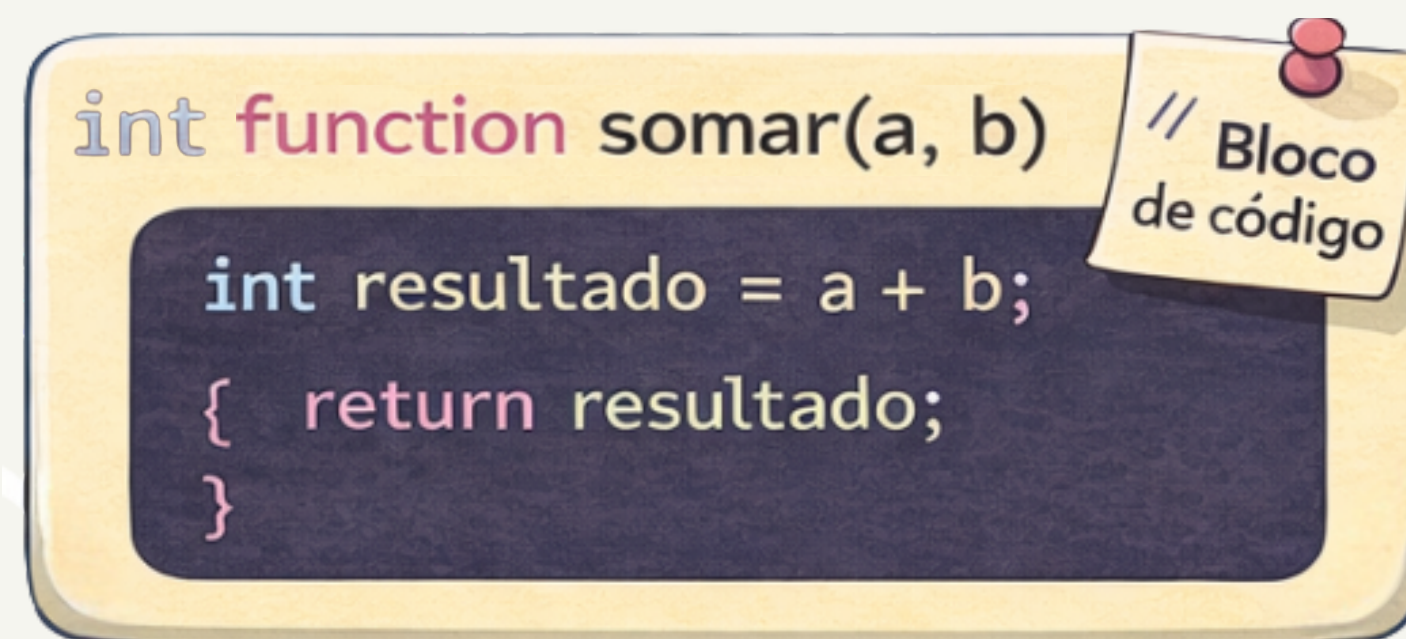
Ex: Imprimir os números de 1 a 10 e pular o 6:

```
int i;  
for(i=1; i<=10; i++){  
    if(i == 6){  
        continue; //Pula a iteracao  
    }  
    printf("%d\n", i);  
}
```

1
2
3
4
5
7
8
9
10

FUNÇÕES

- **Bloco de código** que realiza uma tarefa específica.
- Pode devolver um **valor** como resultado ou **apenas executar uma ação**.
- Torna o código mais **organizado**, fácil de entender e reutilizável.



```
int function somar(a, b)  
  
    int resultado = a + b;  
    { return resultado;  
    }
```

// Bloco de código

FUNÇÕES

Sintaxe:

o que a função devolve

```
tipo_de_retorno nome_da_funcao(parametros)
{
    //bloco de código
};
```

FUNÇÕES

Funções sem retorno: Função apenas executando **ações**.

```
void mostrar_mensagem(){  
    printf("Hello World!\n");  
}
```



Sem o return

FUNÇÕES

Funções sem retorno: Função apenas executando **ações**.

```
void mostrar_mensagem(){  
    printf("Hello World!\n");  
}
```



Sem o return

Funções com retorno: obrigatório o uso do “**return**”

```
float subtracao_dois_numeros(float num1, float num2)  
{  
    return (num1 - num2);  
}
```

ESTRUTURAS DE DADOS



- Sem estrutura definida
- Bagunça de informações

ESTRUTURAS DE DADOS

Formas organizadas de **armazenar, gerenciar, manipular dados** na memória do computador

Informações organizadas acessadas e processadas **eficientemente**

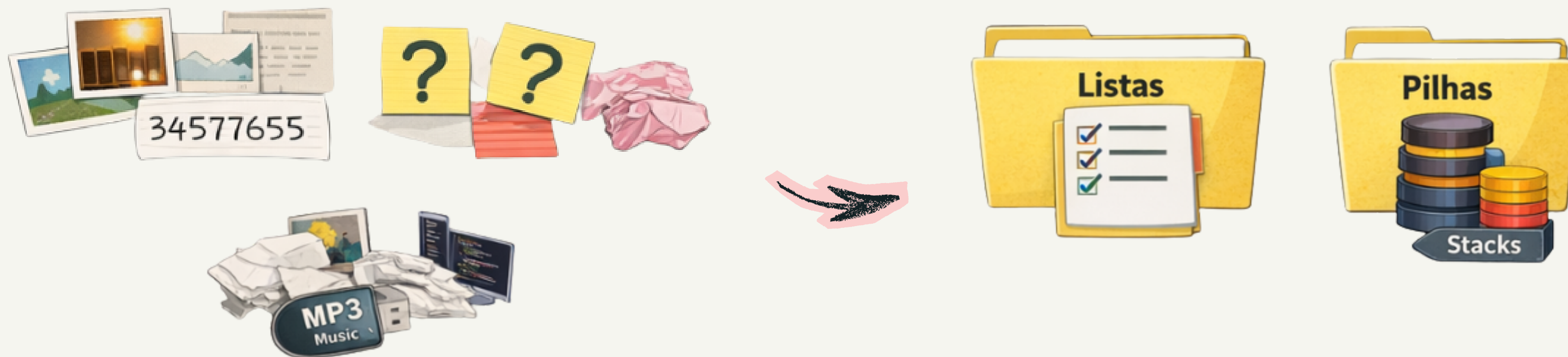


- Sem estrutura definida
- Bagunça de informações

ESTRUTURAS DE DADOS

Formas organizadas de **armazenar, gerenciar, manipular dados** na memória do computador

Informações organizadas acessadas e processadas **eficientemente**



- Sem estrutura definida
- Bagunça de informações



Busca, acesso, inserção

ESTRUTURAS DE DADOS

Como escolher a melhor estrutura de dados?

Tipo de dado e volume

- números, textos?
- Pouco ou muito?

ESTRUTURAS DE DADOS

Como escolher a melhor estrutura de dados?

Tipo de dado e volume

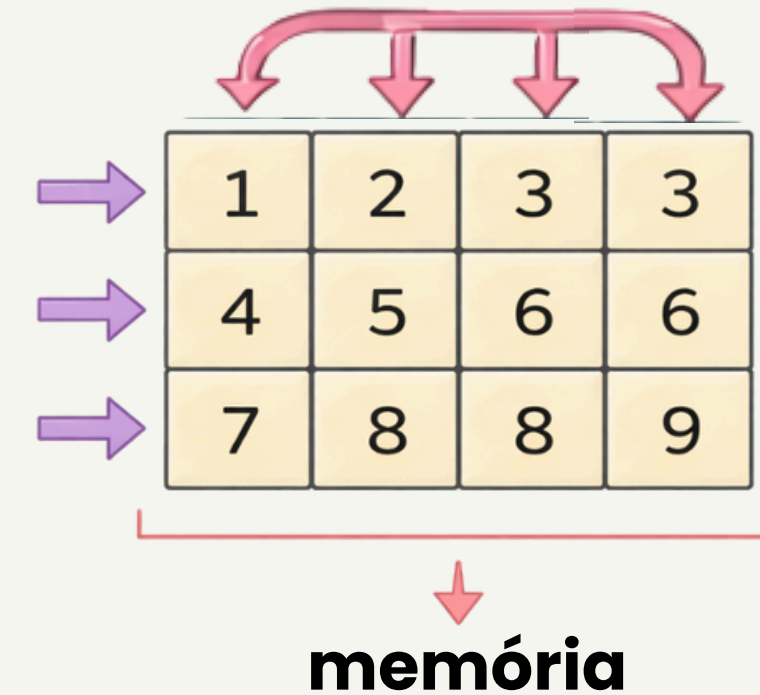
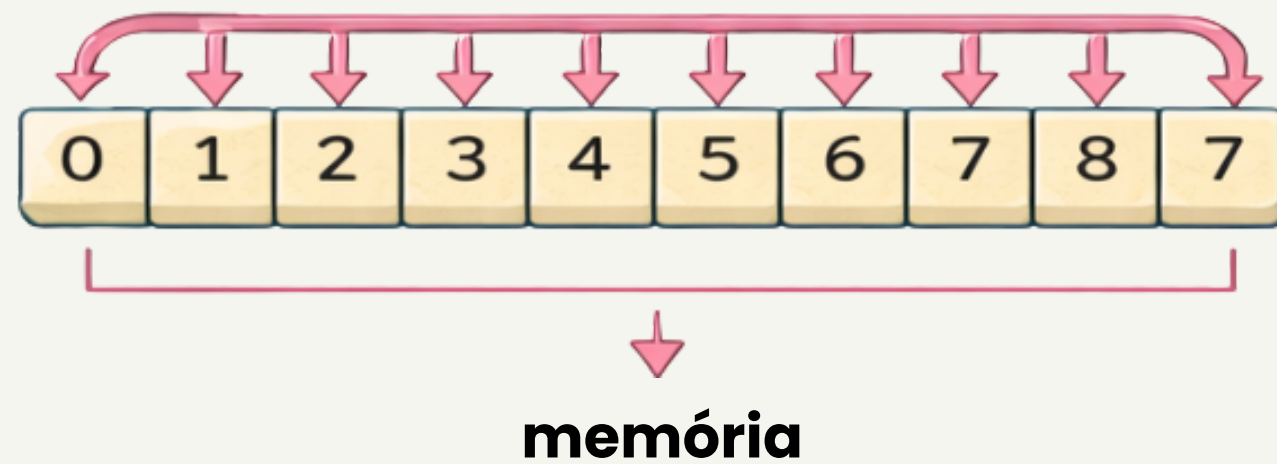
- números, textos?
- Pouco ou muito?

Operações necessárias

- Inserir/remover rápido?
- Acessar diretamente um elemento?
- Armazenar em hierarquia?

Eficiência x Memória

ESTRUTURAS LINEARES ESTÁTICAS



- **Linear:** Ordem sequencial
- **Estáticas:** Tamanho fixo
- **Homogêneas:** elementos do mesmo tipo
- **Memoria contígua:** elementos lado a lado
- **Acesso rápido** pelos índices

Exemplo clássico: ARRAYS

ARRAYS UNIDIMENSIONAIS (VETORES)

Estrutura de dados que permite armazenar vários valores do **mesmo tipo**, organizados em posições consecutivas de memória e acessados por meio de **índices (início em 0)**.

ARRAYS UNIDIMENSIONAIS (VETORES)

Estrutura de dados que permite armazenar vários valores do **mesmo tipo**, organizados em posições consecutivas de memória e acessados por meio de **índices (início em 0)**.

Sintaxe: **tipo** **nome**[**tamanho**];

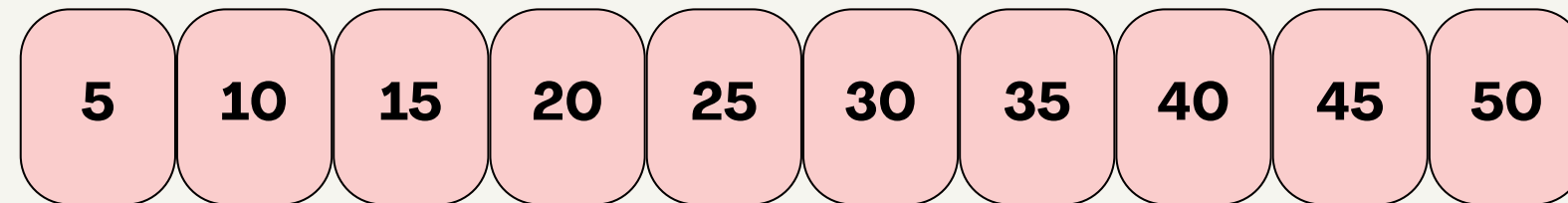
ARRAYS UNIDIMENSIONAIS (VETORES)

Estrutura de dados que permite armazenar vários valores do **mesmo tipo**, organizados em posições consecutivas de memória e acessados por meio de **índices (início em 0)**.

Sintaxe: **tipo** **nome**[**tamanho**];

Declaração:

```
int numeros[10];
```

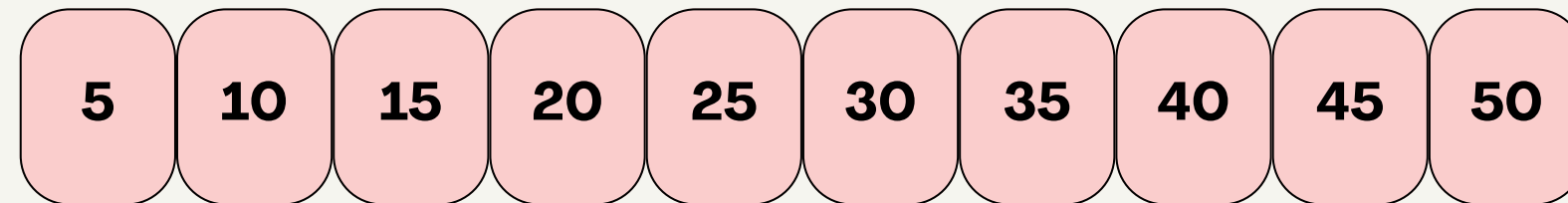


ARRAYS UNIDIMENSIONAIS (VETORES)

Estrutura de dados que permite armazenar vários valores do **mesmo tipo**, organizados em posições consecutivas de memória e acessados por meio de **índices (início em 0)**.

Sintaxe: **tipo nome[tamanho];**

Declaração: `int numeros[10];`



Inserção de valores:

```
numeros[0] = 5; numeros[1] = 10; numeros[2] = 15;  
numeros[3] = 20; numeros[4] = 25;  
numeros[5] = 30; numeros[6] = 35; numeros[7] = 40;  
numeros[8] = 45; numeros[9] = 50;
```

ARRAYS UNIDIMENSIONAIS (VETORES)

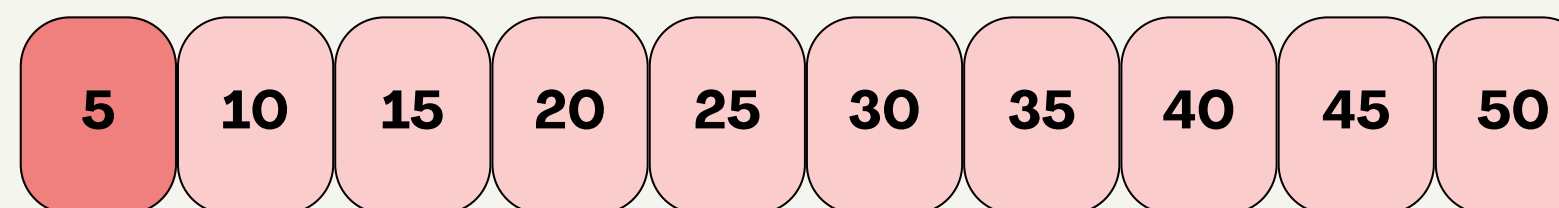
Estrutura de dados que permite armazenar vários valores do **mesmo tipo**, organizados em posições consecutivas de memória e acessados por meio de **índices (início em 0)**.

Sintaxe:

tipo **nome**[**tamanho**];

Declaração:

```
int numeros[10];
```



Inserção de valores:

```
numeros[0] = 5; numeros[1] = 10; numeros[2] = 15;
numeros[3] = 20; numeros[4] = 25;
numeros[5] = 30; numeros[6] = 35; numeros[7] = 40;
numeros[8] = 45; numeros[9] = 50;
```

Acesso aos elementos:

```
printf("O primeiro número do vetor é: %d\n", numeros[0]);
```

ARRAYS UNIDIMENSIONAIS (VETORES)

```
//vetor de notas dos alunos  
float notas[] = {7.5, 8.0, 6.5, 9.0};  
int i;
```

ARRAYS UNIDIMENSIONAIS (VETORES)

```
//vetor de notas dos alunos  
float notas[] = {7.5, 8.0, 6.5, 9.0};  
int i;
```

Percorrendo todos os elementos

for

```
for(i=0; i<4; i++){  
    printf("Nota do aluno %d: %.1f\n", i+1, notas[i]);  
}
```

while

```
while(i<4){  
    printf("Nota do aluno %d: %.1f\n", i+1, notas[i]);  
    i++;  
}
```

*do/
while*

```
do{  
    printf("Nota do aluno %d: %.1f\n", i+1, notas[i]);  
    i++;  
} while(i<4);
```

ARRAYS UNIDIMENSIONAIS (VETORES)

```
//vetor de notas dos alunos  
float notas[] = {7.5, 8.0, 6.5, 9.0};  
int i;
```

Percorrendo todos os elementos

for

```
for(i=0; i<4; i++){  
    printf("Nota do aluno %d: %.1f\n", i+1, notas[i]);  
}
```

while

```
while(i<4){  
    printf("Nota do aluno %d: %.1f\n", i+1, notas[i]);  
    i++;  
}
```

do/
while

```
do{  
    printf("Nota do aluno %d: %.1f\n", i+1, notas[i]);  
    i++;  
} while(i<4);
```

Saída:

```
Nota do aluno 0: 7.5  
Nota do aluno 1: 8.0  
Nota do aluno 2: 6.5  
Nota do aluno 3: 9.0
```

ARRAYS BIDIMENSIONAIS (MATRIZES)

- Matrizes são estruturas de dados **2D**, formadas por **vetores de vetores** em C (tabelas).
- Usadas para organizar informações em **linhas** e **colunas**;
- **Tabelas, planilhas, mapas e matrizes matemáticas.**

ARRAYS BIDIMENSIONAIS (MATRIZES)

- Matrizes são estruturas de dados **2D**, formadas por **vetores de vetores** em C (tabelas).
- Usadas para organizar informações em **linhas** e **colunas**;
- Tabelas, planilhas, mapas e matrizes matemáticas.

Sintaxe: **tipo nome[linhas][colunas];**

ARRAYS BIDIMENSIONAIS (MATRIZES)

- Matrizes são estruturas de dados **2D**, formadas por **vetores de vetores** em C (tabelas).
- Usadas para organizar informações em **linhas** e **colunas**;
- Tabelas, planilhas, mapas e matrizes matemáticas.

Sintaxe: **tipo nome[linhas][colunas];**

```
int matriz[2][3];
```

Declaração

MATRIZ (2x3)	coluna [0]	coluna [1]	coluna [2]
linha [0]			
linha [1]			

ARRAYS BIDIMENSIONAIS (MATRIZES)

Inserção de valores:

```
matriz[0][0] = 1;
matriz[0][1] = 2;
matriz[0][2] = 3;

matriz[1][0] = 4;
matriz[1][1] = 5;
matriz[1][2] = 6;
```

MATRIZ (2x3)	coluna [0]	coluna [1]	coluna [2]
linha [0]	1	2	3
linha [1]	4	5	6

ARRAYS BIDIMENSIONAIS (MATRIZES)

Inserção de valores:

```
matriz[0][0] = 1;
matriz[0][1] = 2;
matriz[0][2] = 3;

matriz[1][0] = 4;
matriz[1][1] = 5;
matriz[1][2] = 6;
```

Declaração e inserção

```
int matriz[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

MATRIZ (2x3)	coluna [0]	coluna [1]	coluna [2]
linha [0]	1	2	3
linha [1]	4	5	6

ARRAYS BIDIMENSIONAIS (MATRIZES)

MATRIZ (2x3)	coluna [0]	coluna [1]	coluna [2]
linha [0]	1	2	3
linha [1]	4	5	6

Acessando
um valor:

ARRAYS BIDIMENSIONAIS (MATRIZES)

MATRIZ (2x3)	coluna [0]	coluna [1]	coluna [2]
linha [0]	1	2	3
linha [1]	4	5	6

Acessando
um valor:

ARRAYS BIDIMENSIONAIS (MATRIZES)

MATRIZ (2x3)	coluna [0]	coluna [1]	coluna [2]
linha [0]	1	2	3
linha [1]	4	5	6

Acessando
um valor:

matriz[1][0]

linha coluna

ARRAYS BIDIMENSIONAIS (MATRIZES)

MATRIZ (2x3)	coluna [0]	coluna [1]	coluna [2]
linha [0]	1	2	3
linha [1]	4	5	6

**Percorrendo
a matriz:**

```
for(int i=0; i<2; i++){//Percorre as linhas da matriz
    for(int j=0; j<3; j++){//Percorre as colunas da matriz
        printf("%d ", matriz[i][j]);
    }
    printf("\n"); //Pula para a proxima linha apos
    imprimir cada linha da matriz
}
```

STRINGS

Vetor de caracteres **char[]** terminado pelo caractere nulo **\0**

Declaração direta

```
char nome[] = "Darcy";
```

**Cria um array com 6 posições:
'D', 'a', 'r', 'c', 'y', '\0'.**

STRINGS

Vetor de caracteres **char[]** terminado pelo caractere nulo **\0**

Declaração direta

```
char nome[] = "Darcy";
```

**Cria um array com 6 posições:
'D', 'a', 'r', 'c', 'y', '\0'.**

**Declaração com
tamanho fixo**

```
char nome[10] = "Moreira";
```

**Cria um array com 10 posições, mas só 8 são usadas:
'M', 'o', 'r', 'e', 'i', 'r', 'a', '\0'.**

BIBLIOTECA <STRING.H>

Trata strings como arrays de caracteres terminados por ‘\0’

BIBLIOTECA <STRING.H>

Trata strings como arrays de caracteres terminados por '\0'

Principais funções:

```
char nome[] = "Harry Potter";  
int tamanho_string = strlen(nome);  
printf("Tamanho: %d\n", tamanho_string); //Tamanho: 12
```

strlen(string) → retorna o número de caracteres, sem contar o \0

BIBLIOTECA <STRING.H>

Trata strings como arrays de caracteres terminados por '\0'

Principais funções:

```
char nome[] = "Harry Potter";  
int tamanho_string = strlen(nome);  
printf("Tamanho: %d\n", tamanho_string); //Tamanho: 12
```

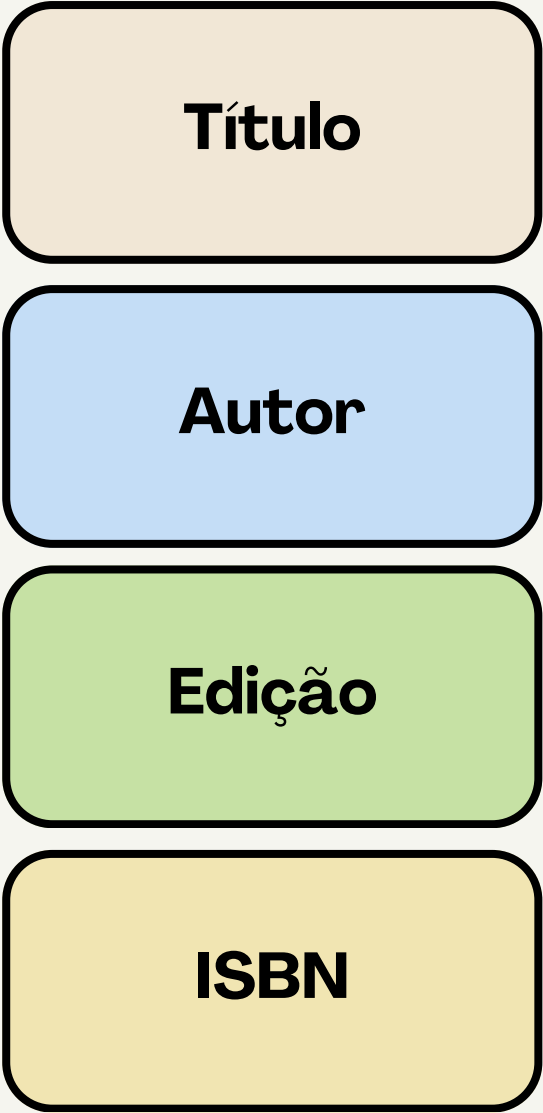
strlen(string) → retorna o número de caracteres, sem contar o \0

```
char nome[20];  
strcpy(nome, "Harry");  
printf("Nome: %s\n", nome); //Nome: Harry
```

strcpy(destino, origem) → Copia uma string pra outra

STRUCTS

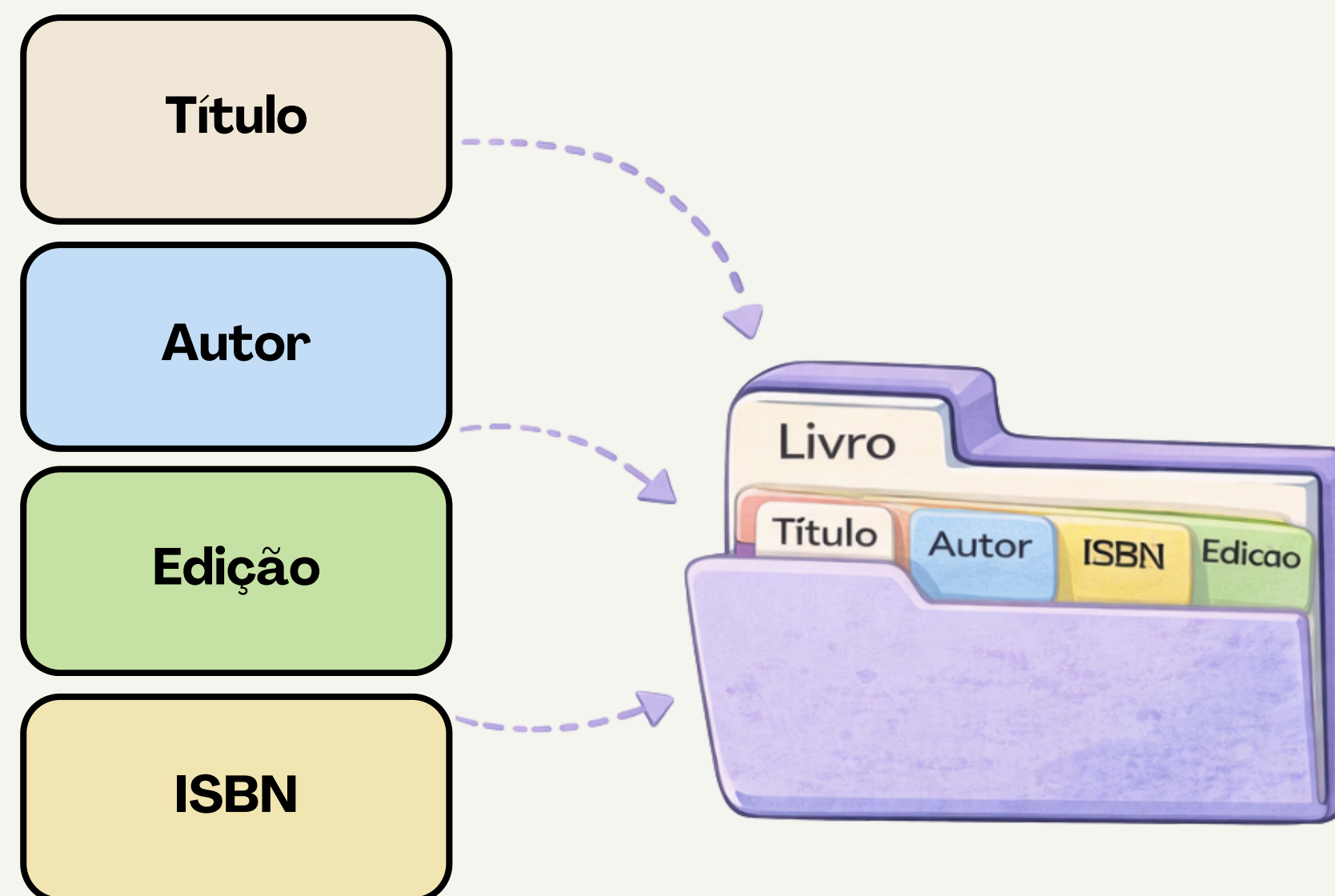
LIVRO



STRUCTS

Permite agrupar diferentes campos de um objeto em uma única variável

LIVRO



STRUCTS

```
struct nome_da_struct {  
    tipo campo1;  
    tipo campo2;  
    ...  
};
```

```
struct Livro // definindo uma struct chamada Livro  
{  
    char titulo[50];  
    char autor[50];  
    int isbn;  
    int edicao;  
};
```

```
struct Livro Livro1; // declarando uma variável do tipo struct Livro
```

STRUCTS

```
typedef struct {  
    tipo campo1;  
    tipo campo2;  
    ...  
} nome_da_struct;
```

STRUCTS

```
typedef struct {  
    tipo campo1;  
    tipo campo2;  
    ...  
} nome_da_struct;
```

typedef é uma palavra **reservada** usada para criar um **novo nome** para um tipo existente

STRUCTS

```
typedef struct {  
    tipo campo1;  
    tipo campo2;  
    ...  
} nome_da_struct;
```

typedef é uma palavra **reservada** usada para criar um **novo nome** para um tipo existente

```
typedef struct  
{  
    char titulo[50];  
    char autor[50];  
    int isbn;  
    int edicao;  
} Livro;
```

Livro Livro1; // declarando uma variável do tipo Livro

STRUCTS

Declarando e atribuindo valor/acessando os campos

```
typedef struct
{
    char titulo[50];
    char autor[50];
    int isbn;
    int edicao;
}Livro;
```

STRUCTS

Declarando e atribuindo valor/acessando os campos

```
typedef struct
{
    char titulo[50];
    char autor[50];
    int isbn;
    int edicao;
}Livro;
```

```
Livro Livro1; // declarando uma variável do tipo struct Livro
/* acessando os campos da estrutura Livro */
strcpy(Livro1.titulo, "O Senhor dos Aneis: As Duas Torres");// atribuindo valor ao campo titulo
Livro1.isbn = 978859;// atribuindo valor ao campo isbn
Livro1.edicao = 3;// atribuindo valor ao campo edicao
strcpy(Livro1.autor, "J.R.R. Tolkien");// atribuindo valor ao campo autor

printf("Detalhes do Livro1:\n");
printf("Titulo: %s\n", Livro1.titulo);
printf("Autor: %s\n", Livro1.autor);
printf("ISBN: %d\n", Livro1.isbn);
printf("Edicao: %d\n", Livro1.edicao);
```

STRUCTS

Declarando e Inicializando uma Struct

```
typedef struct
{
    char titulo[50];
    char autor[50];
    int isbn;
    int edicao;
}Livro;
```

```
// declarando e inicializando uma variavel do tipo struct Livro
Livro Livro1 = {"O Senhor dos Aneis: As Duas Torres", "Tolkien", 12345, 2};
```

STRUCTS

Structs como parâmetros de funções

```
void exibir(Livro L)
{
    printf("Titulo: %s\n", L.titulo);
    printf("Autor: %s\n", L.autor);
    printf("ISBN: %d\n", L.isbn);
    printf("Edicao: %d\n", L.edicao);
}
```

Cópia da struct é feita, logo, as alterações **não** afetam a original.

MODULARIZAÇÃO

Dividir o código fonte em múltiplos arquivos para facilitar a organização, manutenção, reutilização e legibilidade do programa

MODULARIZAÇÃO

Dividir o código fonte em múltiplos arquivos para facilitar a organização, manutenção, reutilização e legibilidade do programa

Header guards: evitam múltiplas inclusões

operacoes.h

```
#ifndef OPERACOES_H
#define OPERACOES_H

int soma(int x, int y);

#endif
```

Declarações

MODULARIZAÇÃO

Dividir o código fonte em múltiplos arquivos para facilitar a organização, manutenção, reutilização e legibilidade do programa

Header guards

```
operacoes.h

#ifndef OPERACOES_H
#define OPERACOES_H

int soma(int x, int y);

#endif
```

Declarações

```
operacoes.c

#include "operacoes.h"

int soma(int x, int y) {
    return x + y;
}
```

Implementações

MODULARIZAÇÃO

Dividir o código fonte em múltiplos arquivos para facilitar a organização, manutenção, reutilização e legibilidade do programa

Header guards

operacoes.h

```
#ifndef OPERACOES_H
#define OPERACOES_H

int soma(int x, int y);

#endif
```

Declarações

operacoes.c

```
#include "operacoes.h"

int soma(int x, int y) {
    {
        return x + y;
    }
}
```

Implementações

main.c

```
#include "operacoes.h"

int main()
{
    int a = 5, b = 3;
    int resultado;

    resultado = soma(a, b);
    printf("%d\\", resultado);
    return 0;
}
```

Chamada das funções

PONTEIROS

São variáveis que armazenam o **endereço de memória** de outras variáveis. Dizemos que um ponteiro **“aponta”** para uma variável quando contém seu endereço.

PONTEIROS

São variáveis que armazenam o **endereço de memória** de outras variáveis. Dizemos que um ponteiro **“aponta”** para uma variável quando contém seu endereço.

Declaração:

tipo_da_variavel_apontada *nome_ponteiro

PONTEIROS

São variáveis que armazenam o **endereço de memória** de outras variáveis. Dizemos que um ponteiro **“aponta”** para uma variável quando contém seu endereço.

Declaração:

tipo_da_variavel_apontada *nome_ponteiro



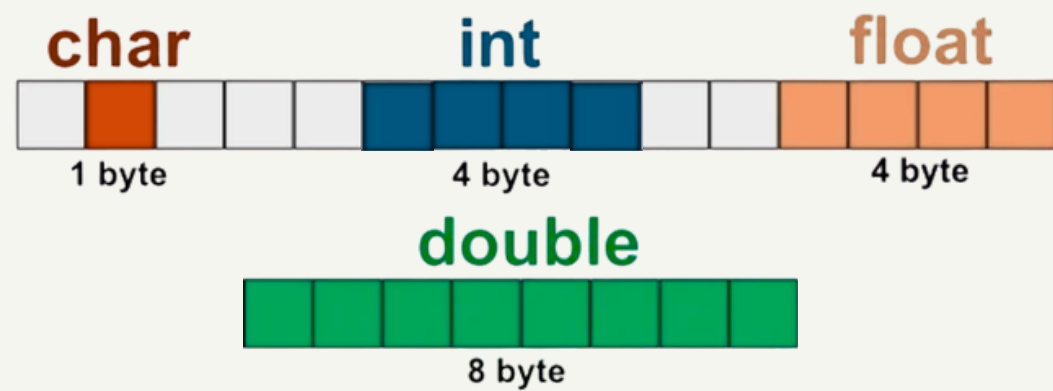
Um bom hábito para evitar problemas de programação é sempre inicializar um ponteiro com **“NULL”**, indicando que ele não aponta para **nenhuma** variável.

```
int *ptr = NULL; // ponteiro nulo
```

PONTEIROS

Inicializando um ponteiro

```
int numero = 5;  
int *ptr_numero;  
ptr_numero = &numero;
```



PONTEIROS

Inicializando um ponteiro

```
int numero = 5;
int *ptr_numero;
ptr_numero = &numero;
```

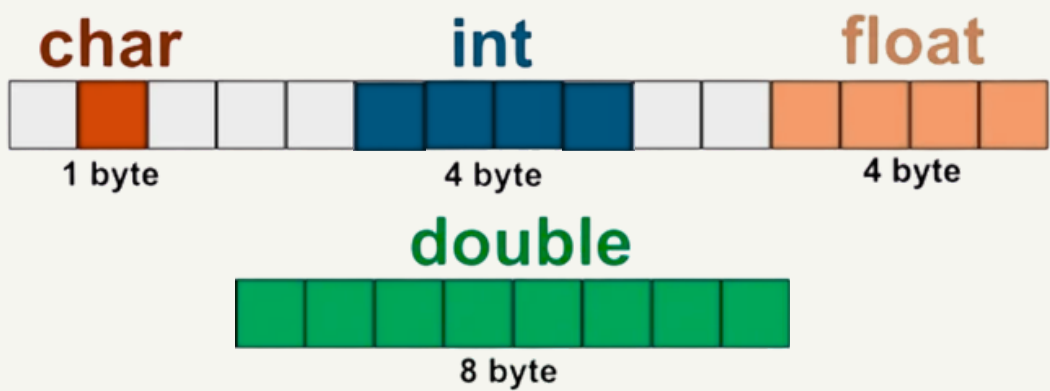
Memória RAM

Endereço de memória Conteúdo da memória

2001	5
2002	
2003	
2004	

int numero

⋮

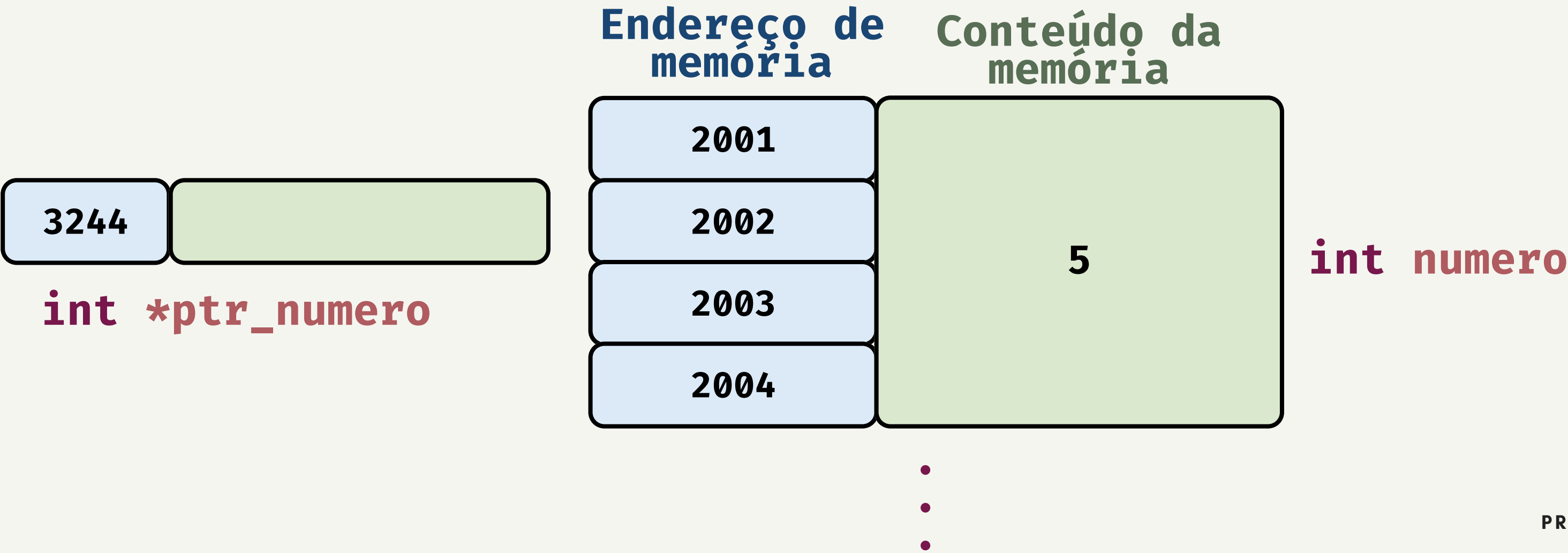


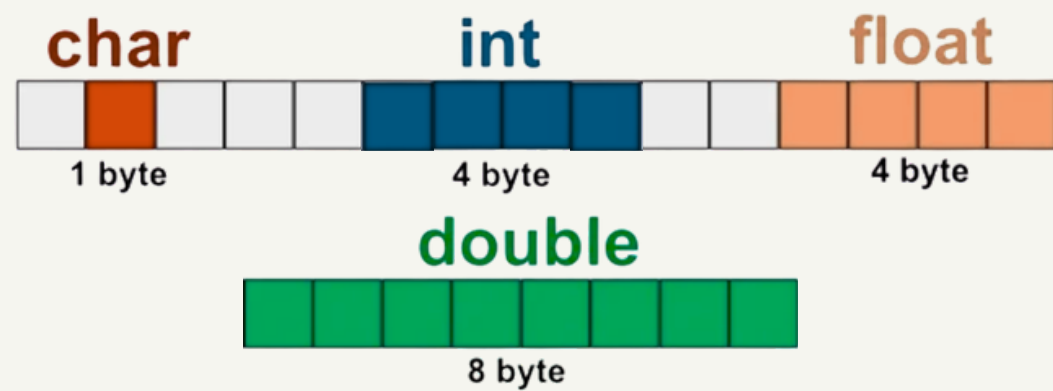
PONTEIROS

Inicializando um ponteiro

```
int numero = 5;
int *ptr_numero;
ptr_numero = &numero;
```

Memória RAM



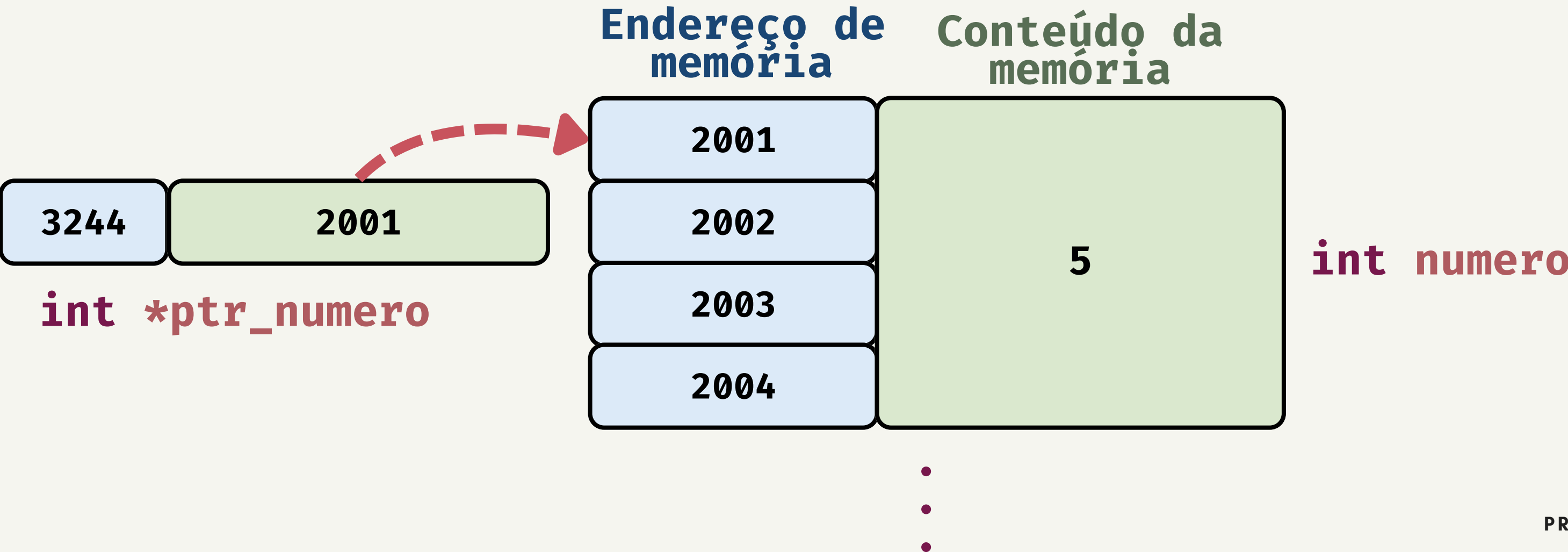


PONTEIROS

Inicializando um ponteiro

```
int numero = 5;  
int *ptr_numero;  
ptr_numero = &numero;
```

Memória RAM



PONTEIROS

variavel → Guarda um **valor**

&variavel → Guarda o **endereço de memória** onde o valor está armazenado

ponteiro; → Guarda um **endereço de memória**

***ponteiro;** → Acessa o **valor** armazenado no endereço de memória guardado

&ponteiro; → Guarda o **endereço de memória** onde o ponteiro está armazenado

```
int numero = 5;  
int *ptr_numero;  
ptr_numero = &numero;
```

PONTEIROS

variavel → Guarda um **valor**

&variavel → Guarda o **endereço de memória** onde o valor está armazenado

ponteiro; → Guarda um **endereço de memória**

***ponteiro;** → Acessa o **valor** armazenado no endereço de memória guardado

&ponteiro; → Guarda o **endereço de memória** onde o ponteiro está armazenado

```
int numero = 5;  
int *ptr_numero;  
ptr_numero = &numero;
```

```
printf("Valor de numero: %d\n", numero); // 5  
printf("Endereço de numero: %p\n", &numero); // 0x7fffe73074d8  
printf("Valor armazenado em ptr_numero: %p\n", ptr_numero); // 0x7fffe73074d8  
printf("Valor apontado por ptr_numero: %d\n", *ptr_numero); // 5  
printf("Endereço de ptr_numero: %p\n", &ptr_numero); // 0x7fffe73074e0
```

PONTEIROS DE VETORES

Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

vetor = **&**v**etor** = **&**v**etor**[0] → endereço do primeiro elemento do vetor

PONTEIROS DE VETORES

Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

vetor = &vetor = &vetor[0] → endereço do primeiro elemento do vetor

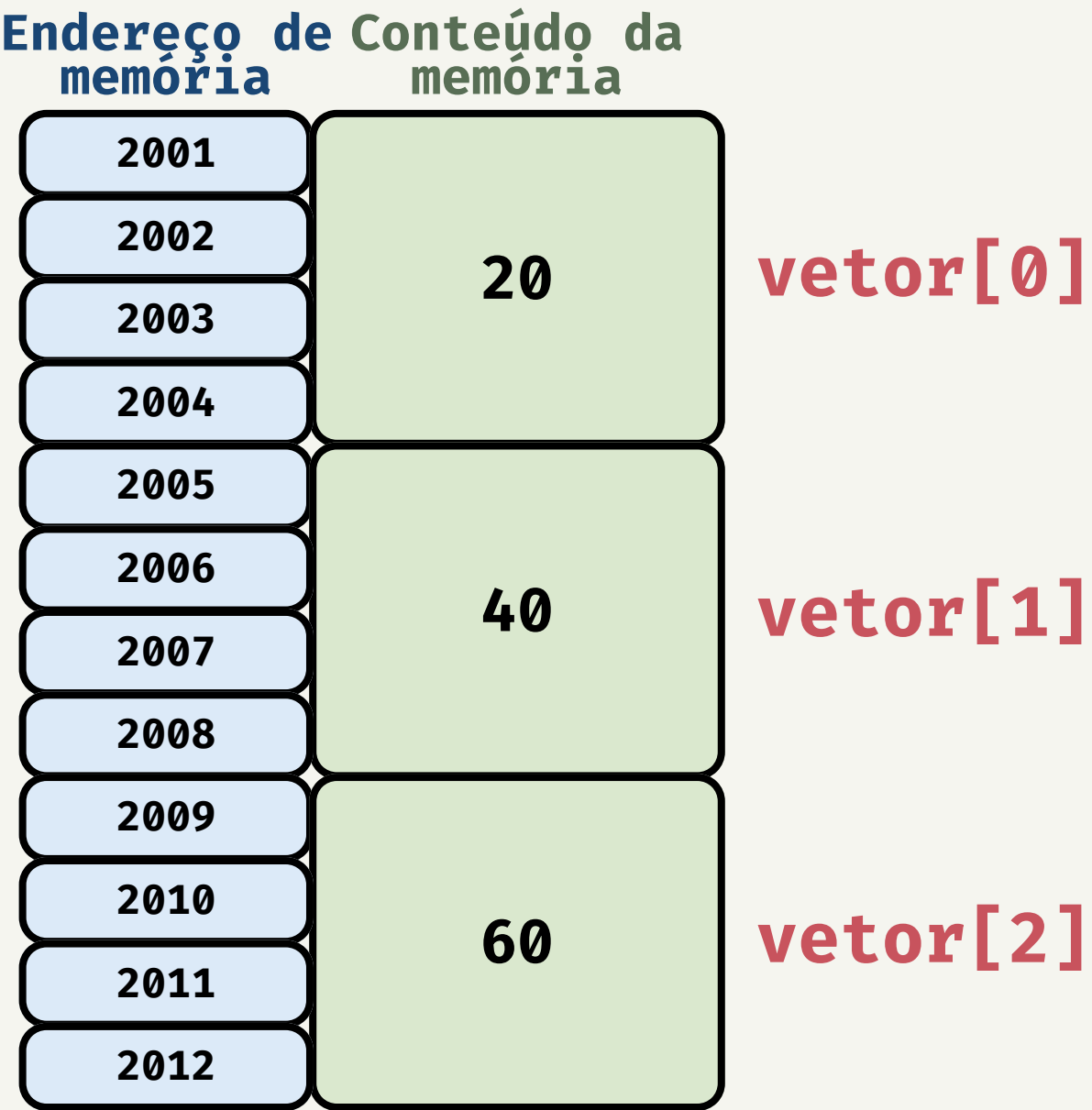
```
int vetor[3] = {20, 40, 60};
```

PONTEIROS DE VETORES

Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

vetor = &vetor = &vetor[0] → endereço do primeiro elemento do vetor

```
int vetor[3] = {20, 40, 60};
```

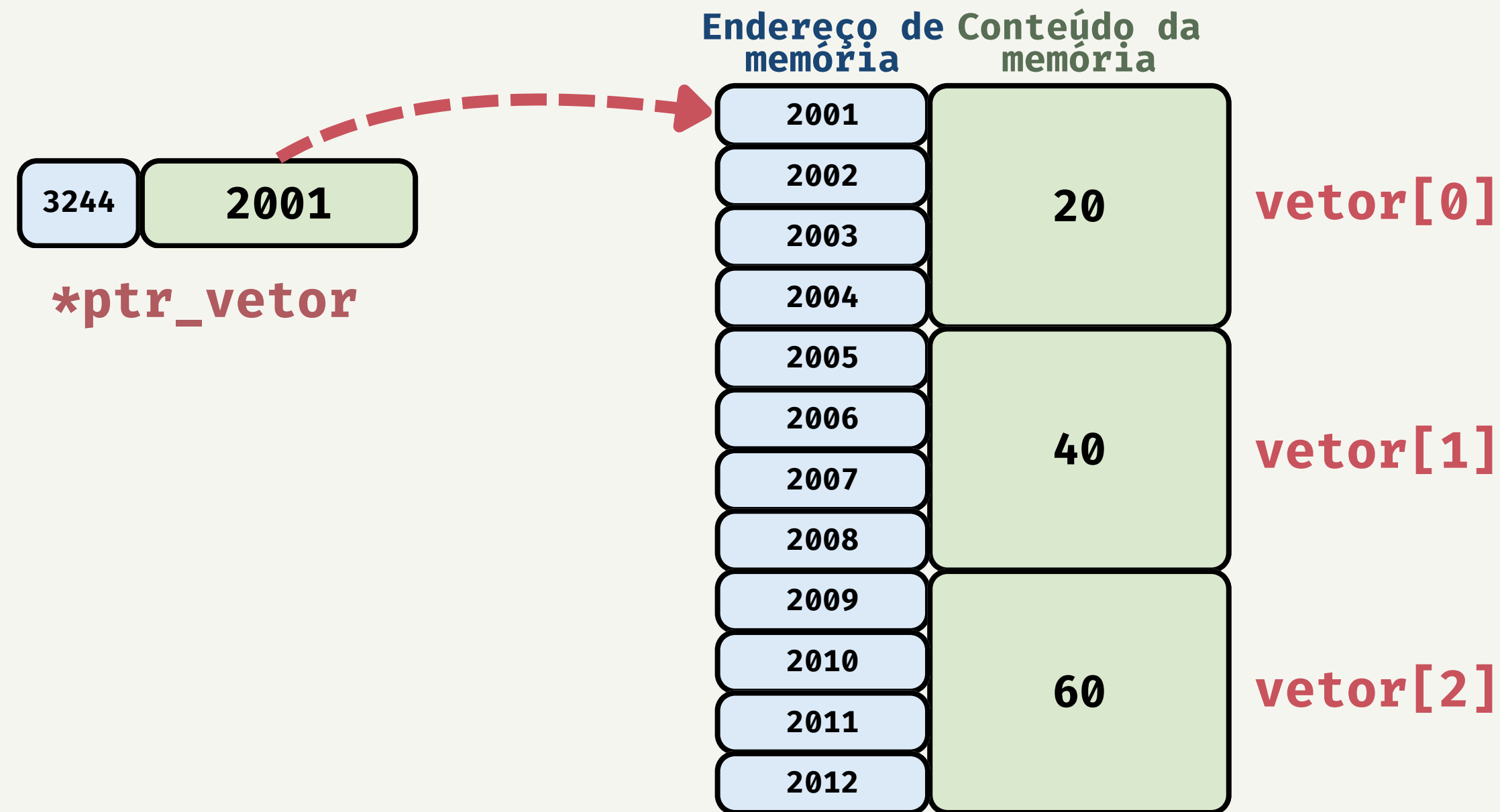


PONTEIROS DE VETORES

Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

vetor = &vetor = &vetor[0] → endereço do primeiro elemento do vetor

```
int vetor[3] = {20, 40, 60};
int *ptr_vetor = vetor;
```



PONTEIROS DE VETORES

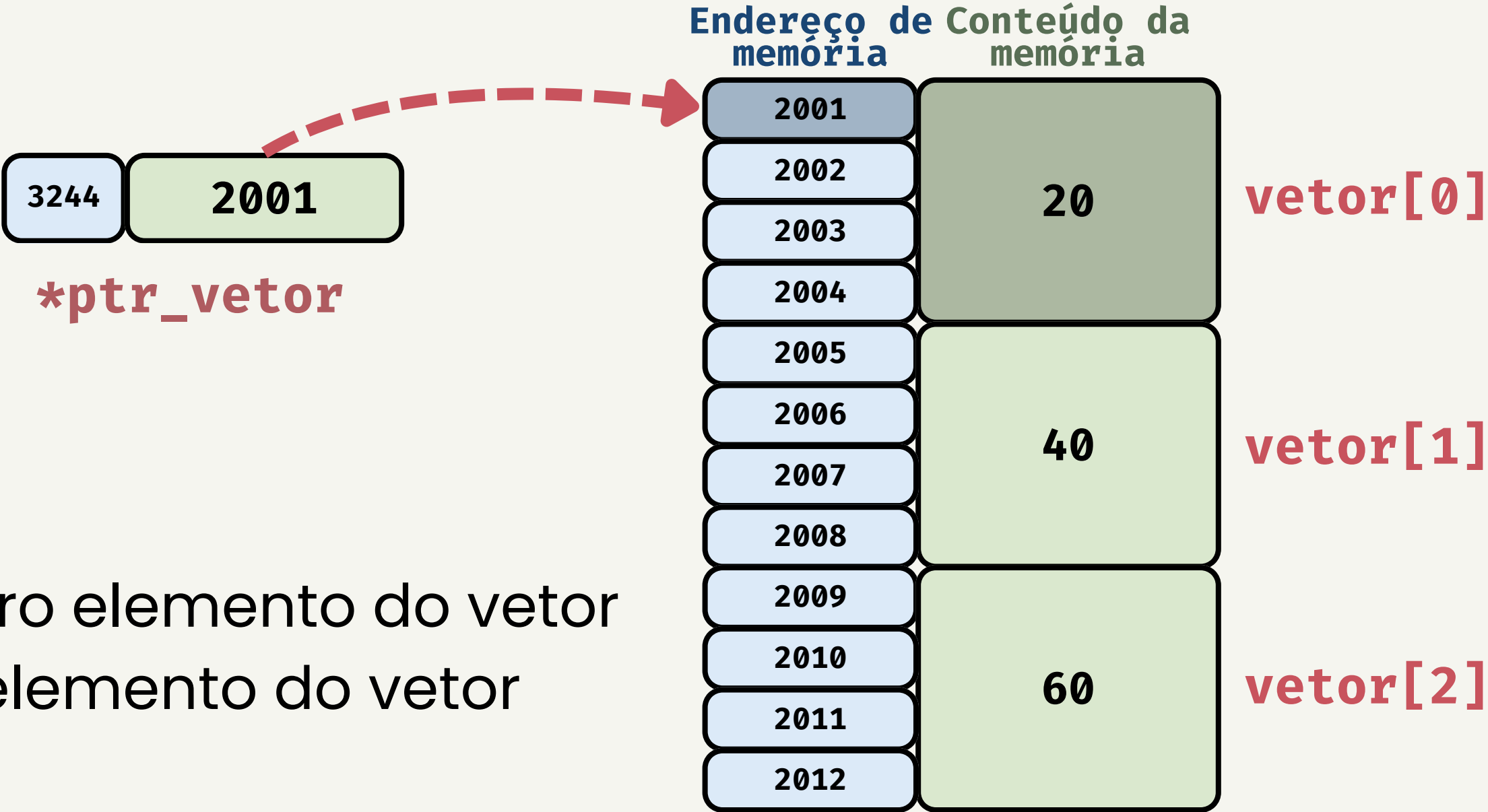
Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

vetor = **&vetor** = **&vetor[0]** → endereço do primeiro elemento do vetor

```
int vetor[3] = {20, 40, 60};
int *ptr_vetor = vetor;
```



ptr_vetor → endereço do primeiro elemento do vetor
***ptr_vetor** → valor do primeiro elemento do vetor



PONTEIROS DE VETORES

Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

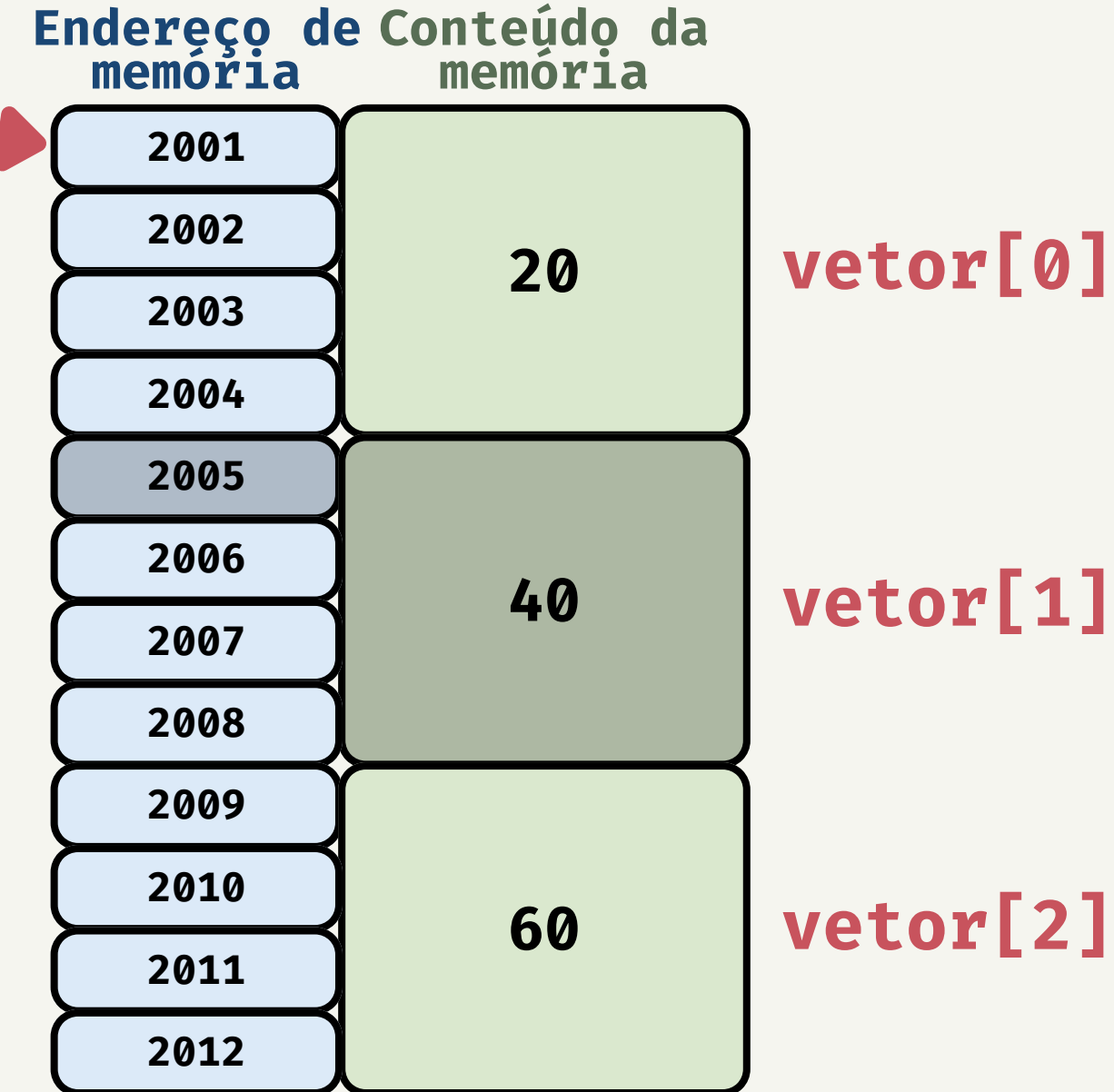
vetor = &vetor = &vetor[0] → endereço do primeiro elemento do vetor

```
int vetor[3] = {20, 40, 60};
int *ptr_vetor = vetor;
```

3244 2001
*ptr_vetor



ptr_vetor + 1; → endereço do segundo elemento do vetor
***(ptr_vetor + 1)** → valor do segundo elemento do vetor



PONTEIROS DE STRINGS

Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

Todos os caracteres podem ser acessados **sequencialmente** até encontrar o `\0`.

```
char string[]="Lua";
```

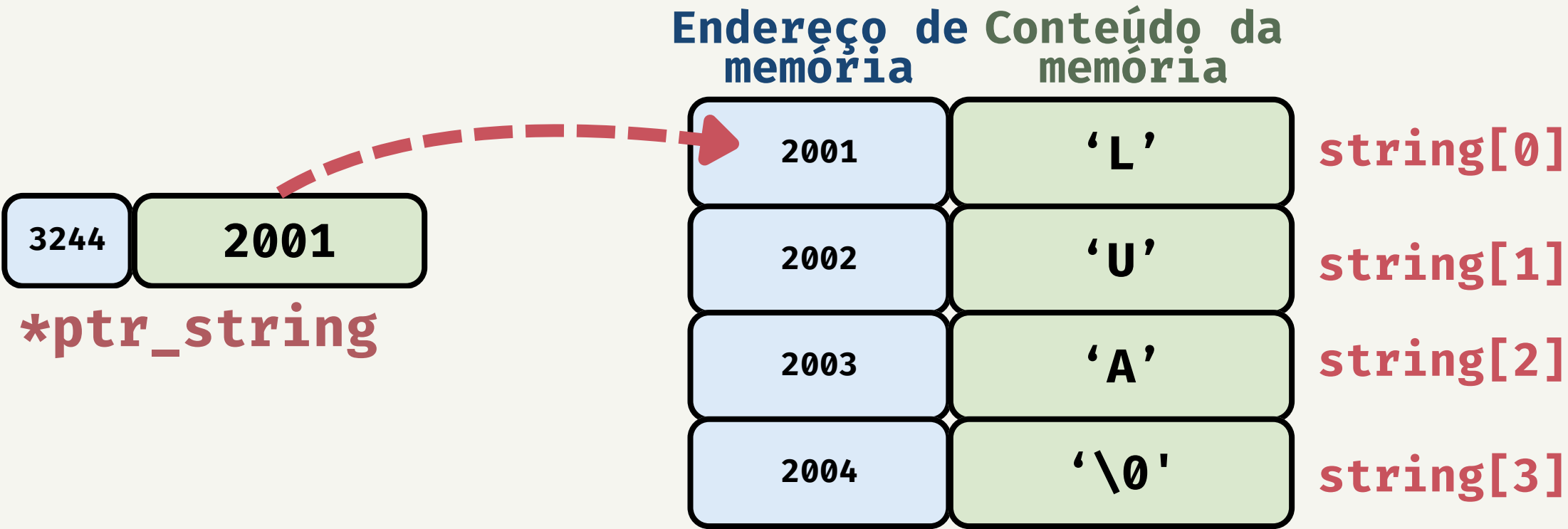
Endereço de memória	Conteúdo da memória	
2001	'L'	string[0]
2002	'U'	string[1]
2003	'A'	string[2]
2004	'\0'	string[3]

PONTEIROS DE STRINGS

Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

Todos os caracteres podem ser acessados **sequencialmente** até encontrar o `\0`.

```
char string[]="Lua";
char *ptr_string = string;
```

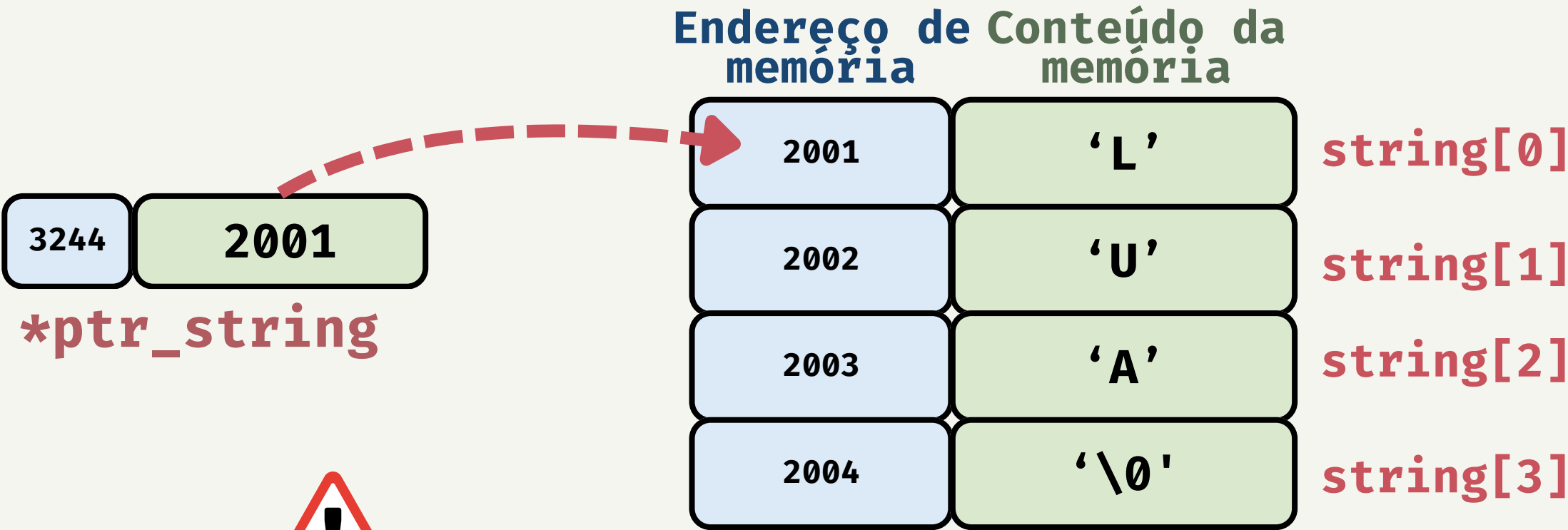


PONTEIROS DE STRINGS

Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

Todos os caracteres podem ser acessados **sequencialmente** até encontrar o `\0`.

```
char string[]="Lua";
char *ptr_string = string;
```



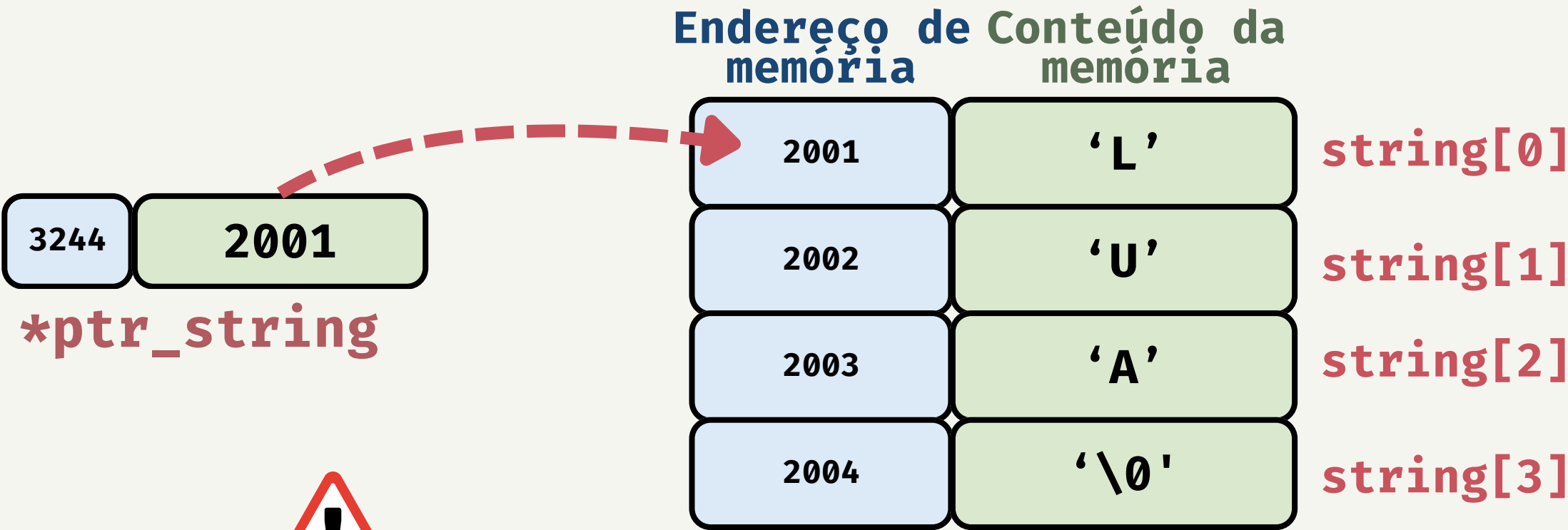
```
printf("String armazenada em ptr_string: %s\n", ptr_string);
printf("Endereço armazenado em ptr_string: %p\n", ptr_string);
```

PONTEIROS DE STRINGS

Um **array** é um **ponteiro** que indica a posição de **memória do primeiro elemento**.

Todos os caracteres podem ser acessados **sequencialmente** até encontrar o `\0`.

```
char string[]="Lua";
char *ptr_string = string;
```



```
printf("String armazenada em ptr_string: %s\n", ptr_string);
printf("Endereço armazenado em ptr_string: %p\n", ptr_string);
```

Saída:

```
String armazenada em ptr_nome: Lua
Endereço armazenado em ptr_nome: 0x7ffca0006
```

PONTEIRO DE STRUCT

```
typedef struct
{
    char titulo[50];
    char autor[50];
    int isbn;
    int edicao;
}Livro;
```

PONTEIRO DE STRUCT

```
typedef struct
{
    char titulo[50];
    char autor[50];
    int isbn;
    int edicao;
}Livro;
```

Acesso a um **campo** da **Struct** utilizando ponteiro é com ->

```
Livro Livro1={"Senhor dos Aneis: As duas torres", "J.R.R. Tolkien", 12345, 2};
Livro *ptr_Livro1; // declarando um ponteiro para a estrutura TLivro
ptr_Livro1 = &Livro1; // atribuindo o endereço da variável Livro ao ponteiro ptr_Livro
printf("Titulo: %s\n", ptr_Livro1->titulo); // acessando o campo titulo usando o ponteiro
printf("Autor: %s\n", ptr_Livro1->autor);    // acessando o campo autor usando o ponteiro
printf("ISBN: %d\n", ptr_Livro1->isbn);      // acessando o campo isbn usando o ponteiro
printf("Edicao: %d\n", ptr_Livro1->edicao);    // acessando o campo edicao usando o ponteiro
```

PASSAGEM POR REFERÊNCIA

Passar o endereço de uma variável (**ponteiro**) para uma **função** permite **modificar** seu valor **globalmente**

```
typedef struct
{
    char titulo[50];
    char autor[50];
    int isbn;
    int edicao;
}Livro;
```

```
void alterarEdicao(Livro *ptr Livro, int novaEdicao)
{
    ptr_Livro->edicao = novaEdicao;
}
```

ALOCAÇÃO DINÂMICA DE MEMÓRIA

ALOCAÇÃO DINÂMICA DE MEMÓRIA

Reserva memória variável durante a execução do programa.

ALOCAÇÃO DINÂMICA DE MEMÓRIA

Reserva memória variável durante a execução do programa.

Fundamental quando:

- O **tamanho dos dados** não é conhecido em tempo de compilação
- Quero usar memória de forma **eficiente**
- Trabalha-se com estruturas **complexas (listas, pilhas, árvores)**

ALOCACAÇÃO DINÂMICA DE MEMÓRIA

Reserva memória variável durante a execução do programa.

Fundamental quando:

- O **tamanho dos dados** não é conhecido em tempo de compilação
- Quero usar memória de forma **eficiente**
- Trabalha-se com estruturas **complexas (listas, pilhas, árvores)**

#include <stdlib.h>

ALOCAÇÃO DINÂMICA DE MEMÓRIA

malloc: Um bloco de memória de 20 bytes é alocado ao ptr

← 20 bytes de memória →

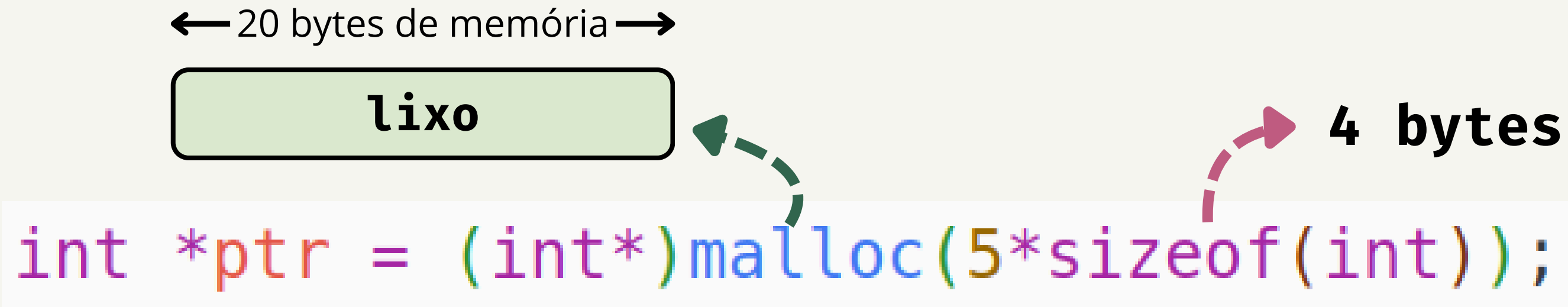
lixo

4 bytes

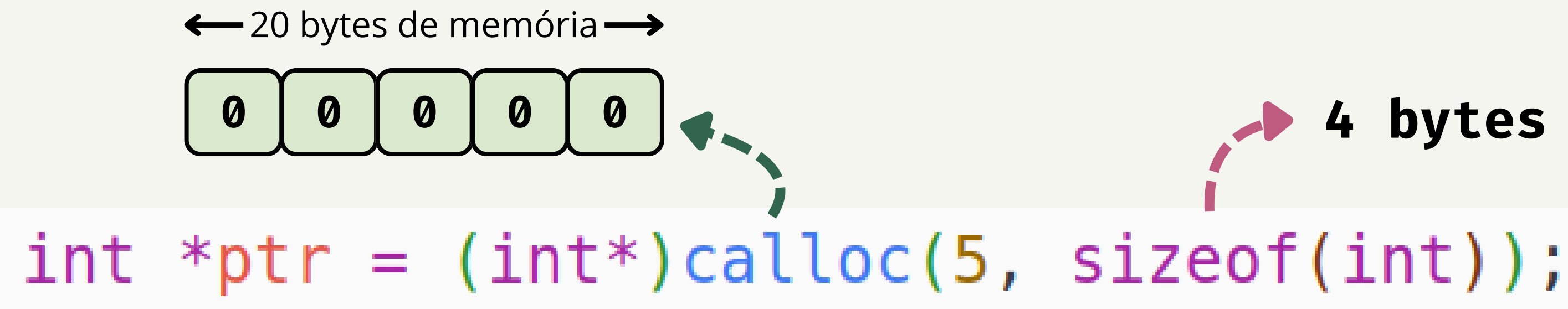
```
int *ptr = (int*)malloc(5*sizeof(int));
```

ALOCAÇÃO DINÂMICA DE MEMÓRIA

malloc: Um bloco de memória de 20 bytes é alocado ao ptr (memória não inicializada)



calloc: 5 blocos de 4 bytes de memória inicializados com zero são alocados ao ptr



ALOCAÇÃO DINÂMICA DE MEMÓRIA

realloc: redimensiona o tamanho de um ponteiro

← 20 bytes de memória →

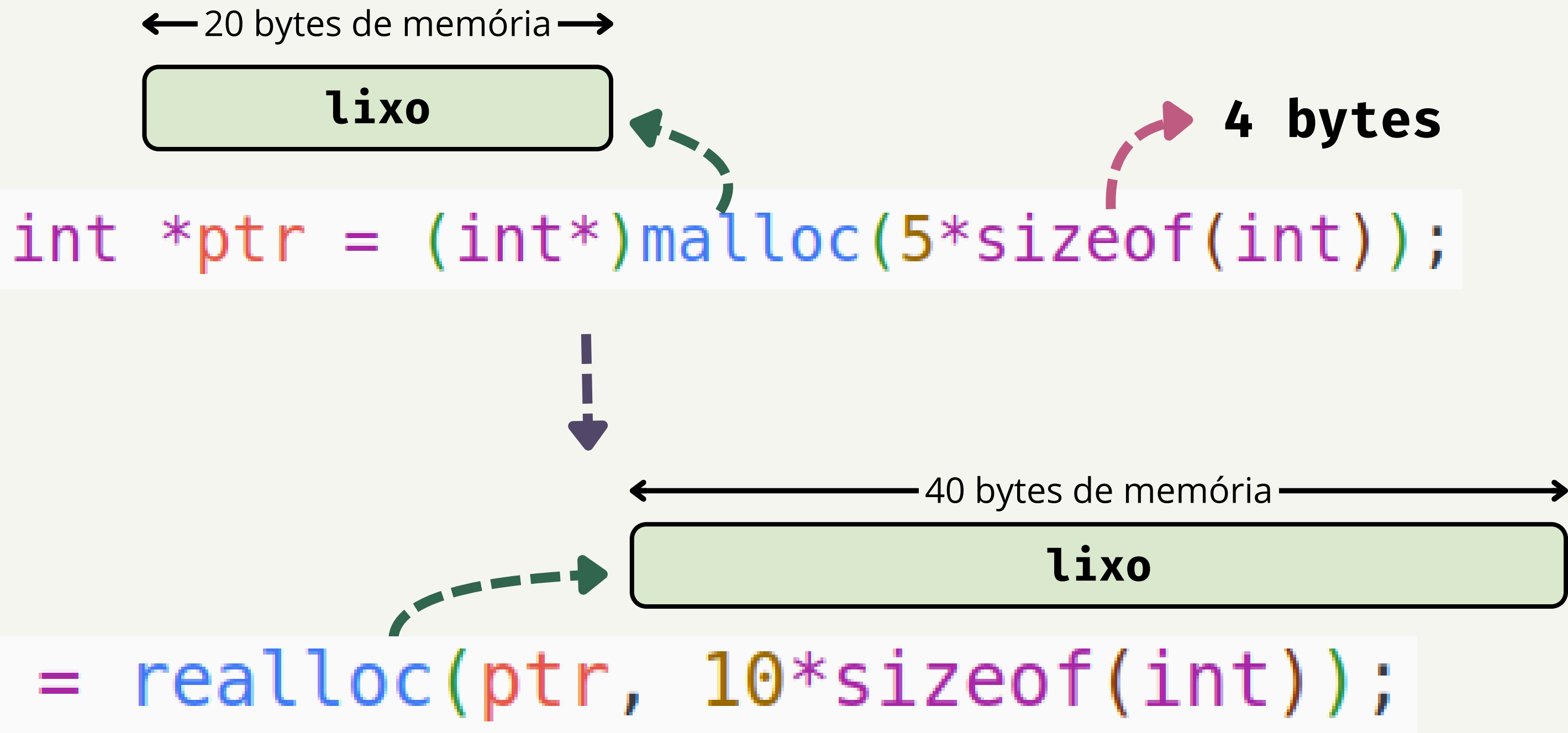
lixo

4 bytes

```
int *ptr = (int*)malloc(5*sizeof(int));
```

ALOCAÇÃO DINÂMICA DE MEMÓRIA

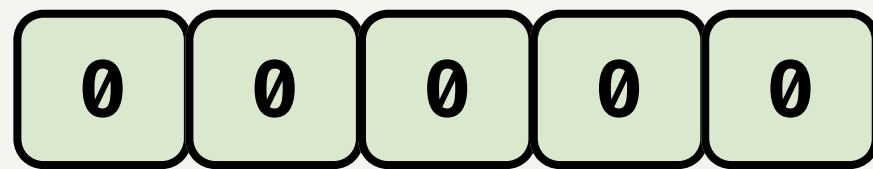
realloc: redimensiona o tamanho de um ponteiro



ALOCAÇÃO DINÂMICA DE MEMÓRIA

free: libera a memória para o sistema

← 20 bytes de memória →



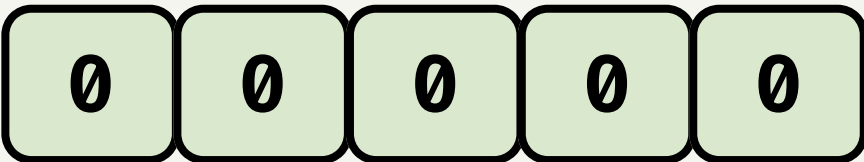
4 bytes

```
int *ptr = (int*)calloc(5, sizeof(int));
```

ALOCAÇÃO DINÂMICA DE MEMÓRIA

free: libera a memória para o sistema

← 20 bytes de memória →

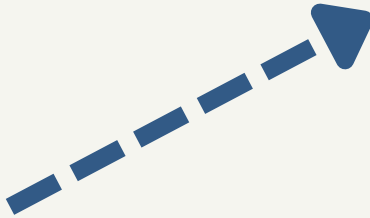
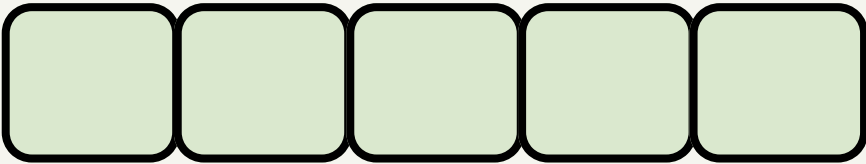
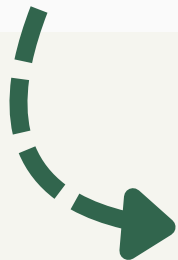


4 bytes

```
int *ptr = (int*)calloc(5, sizeof(int));
```



```
free(ptr);
```



ALOCAÇÃO DINÂMICA DE MEMÓRIA

Boas práticas

ALOCAÇÃO DINÂMICA DE MEMÓRIA

Boas práticas

1. Verificar se malloc()/calloc() retornou NULL

```
int *ptr = (int*)malloc(5*sizeof(int));  
if (ptr == NULL) {  
    printf("Erro na alocação de memória.\n");  
}  
//evita acesso de memória inválido
```

ALOCAÇÃO DINÂMICA DE MEMÓRIA

Boas práticas

1. Verificar se malloc()/calloc() retornou NULL

```
int *ptr = (int*)malloc(5*sizeof(int));  
if (ptr == NULL) {  
    printf("Erro na alocação de memória.\n");  
}  
//evita acesso de memória inválido
```

2. Usar free() para cada malloc()/calloc()

```
int *ptr = (int*)malloc(5*sizeof(int));  
free(ptr);  
// Evita vazamento de memória,  
// mas ptr ainda aponta para a memória liberada
```

ALOCAÇÃO DINÂMICA DE MEMÓRIA

Boas práticas

3. Não acessar fora dos limites alocados

```
int *ptr = (int*)malloc(5*sizeof(int));  
ptr[10] = 100; // Acesso inválido, fora dos limites alocados
```

Comportamento indefinido

ALOCAÇÃO DINÂMICA DE MEMÓRIA

Boas práticas

4. Evitar duplo free() ou free() em ponteiro não alocado:

```
int *ptr;  
free(ptr);  
// Pode causar crash ou bugs difíceis de detectar
```

ALOCAÇÃO DINÂMICA DE MEMÓRIA

Boas práticas

4. Evitar duplo free() ou free() em ponteiro não alocado:

```
int *ptr;  
free(ptr);  
// Pode causar crash ou bugs difíceis de detectar
```

5. Setar o ponteiro como NULL após liberar/free()

```
free(ptr);  
ptr = NULL; // evitar ponteiro pendente
```